



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

CLAUDE/AI AGENT & PIPELINE

LECTURER: DR. ARYATI BINTI BAKRI

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
DAMIYA AINA BINTI BASIR ABD SHAMMAD	A23CS0220

a. What is AI-Assisted Data Engineering

According to data infrastructure paradigms outlined by lakeFS, AI-Assisted Data Engineering represents a fundamental shift from traditional, rigid, rule-based pipeline management to an autonomous, semantic-driven data lifecycle framework. It is the practice of integrating Large Language Models (LLMs) and specialized AI Agents directly into the Extract, Transform, and Load (ETL) pipeline architecture.

Historically, data engineering relied entirely on human developers manually defining schema structures, tracking layout exceptions, writing custom RegEx filters, and mapping data formats. AI-assisted data engineering redefines this by leveraging artificial intelligence to handle the "technical drudgery", the unstructured, unpredictable string variables, mixed data styles, and unexpected log variants allowing engineering teams to scale operational infrastructure seamlessly.

b. What is Ai Agent Used in The Tutorial?

There will be two different methods included in this report since each member followed a different tutorial. Explanation will be done for both to understand what was done to produce the AI-assisted data pipelines.

1. Gemini (Google GenAI)

In this tutorial, our framework elevates Gemini (Google GenAI framework) into an active, autonomous AI Data Engineering Agent. Rather than acting as a passive assistant outside the system, Gemini was integrated directly into the automated execution runtime of the pipeline to improve the data quality of raw dataset. In this tutorial, a Netflix sample dataset with some messy data from Kaggle is used. The agent follows a dynamic loop cycle comprising four core phases:

1. **Observe (Data Profiling):** The pipeline ingests the raw, unclean dataset and automatically constructs a text-based semantic profile including schema shapes, columns, null value distribution, and a slice of rows. It passes this context directly into the agent.
2. **Plan (Quality Assessment):** Gemini reads the profile semantically. Instead of matching text line-by-line, it reasons through data quality discrepancies such as discovering that rows 6, 14, and 20 use mixed-casing like tv show and movie, or noticing that column contains unstandardized text-based and numerical date fragments.
3. **Execute (Dynamic Script Generation):** The agent dynamically engineers a targeted, custom Python Pandas cleaning script tailored specifically to the structural anomalies identified in that specific data batch.

4. **Evaluate (Validation & Load):** The pipeline programmatically executes the agent's code, validates it against a data framework sandbox, converts it to an analytical matrix, and persists it into storage.

2. ChatGPT (AI Assistant)

In this lesson, the K-pop Album Sales Data Engineering Pipeline was developed and improved with the help of ChatGPT, an AI-assisted data engineering assistant. ChatGPT functions as a collaborative assistant that assists the data engineer in designing, implementing, debugging, and documenting the pipeline process, in contrast to a completely autonomous AI agent that runs directly inside the pipeline runtime.

Throughout the data engineering lifecycle, the AI support adheres to a number of crucial tasks:

1. **Planning (Pipeline Design):** ChatGPT assists in designing the overall pipeline architecture by suggesting the implementation of the Medallion Architecture approach (Bronze → Silver → Gold). It helps define the responsibilities of each layer, data flow, folder structure, and processing sequence required for building a structured data engineering workflow.
2. **Code Generation and Development Assistance:** ChatGPT helps generate Python scripts using Pandas for each pipeline component:
 - Bronze layer for raw CSV ingestion
 - Silver layer for data cleaning and feature engineering
 - Gold layer for business analytics aggregation
 - AI Quality Check module for data validation

The generated code is then reviewed, tested, and modified by the data engineer according to the dataset requirements.

3. **Debugging and Problem Solving:** During pipeline execution, ChatGPT assists in identifying and resolving technical issues such as:
 - incorrect column references
 - missing files or paths
 - data type inconsistencies
 - execution errors
 - schema mismatches between pipeline layers

By analyzing error messages and code structure, ChatGPT provides possible solutions and improvements to maintain pipeline reliability.

4. **Data Quality Improvement :** ChatGPT supports the implementation of data validation processes by suggesting checks for:

- missing values
- duplicate records
- invalid numerical values
- inconsistent data formats
- unexpected dataset structures

This improves the reliability of the Silver and Gold layer outputs before generating business insights.

5. **Documentation and Project Enhancement:** ChatGPT assists in producing technical documentation including:

- README documentation
- pipeline explanation
- use case description
- architecture explanation
- report summaries

c. Use Case Diagram

There will be two different use cases included in this report since each member followed a different tutorial. Explanation will be done for both to understand what was done to produce the AI-assisted data pipelines.

1. Gemini (Google GenAI)

This use case diagram in Figure 1 represents an automated AI-assisted data pipeline that functions as an assembly line to clean messy data. On the left side, the Data Source (the user or engineer) kicks off the process by providing the raw Netflix dataset into the pipeline box through Data Extraction, which handles the initial file reading. Inside the pipeline box, the system coordinates three closely linked operations: extracting the data, transforming it, and cleaning it. Instead of relying on a human to write hardcoded rules, the system passes a summary of the data out to the Gemini AI Agent on the right side; Gemini analyzes the problems and sends back a custom cleaning plan during Data Transformation. This plan is then dynamically included during the Data Loading and Data Cleaning steps to automatically fix issues like mixed date formats and inconsistent text casing. Once the cleaning process is complete, the final, standardized data is pushed directly out of the pipeline box to Google Drive Storage, delivering a perfectly polished CSV file back to the user without any manual coding required.

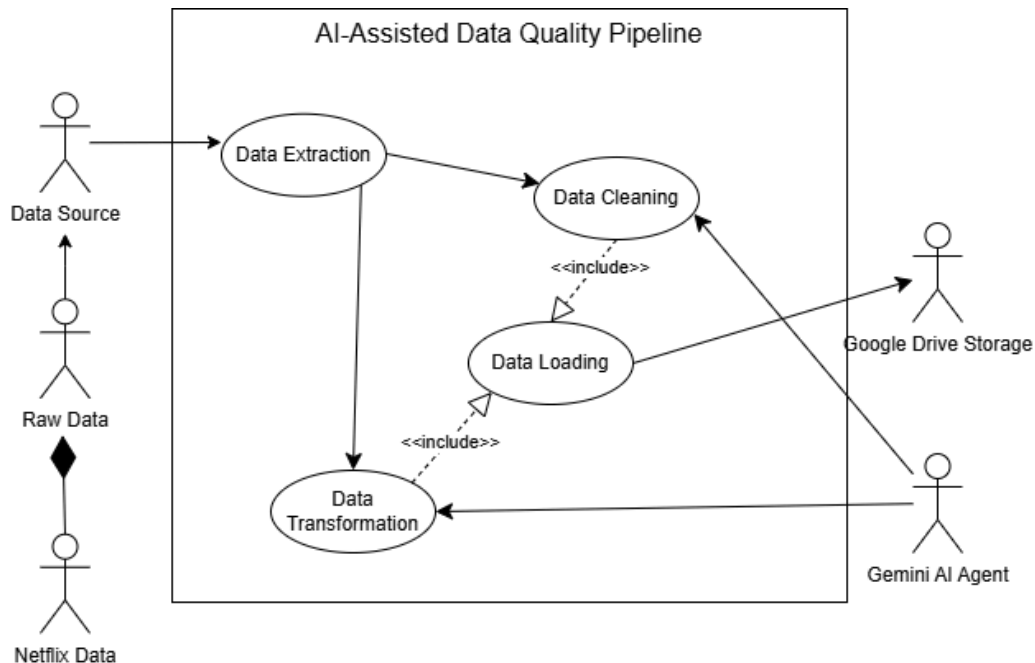


Figure 1: AI-Assisted Data Quality Pipeline Use Case Diagram

2. AI Assistant (CHATGPT)

Figure 2 below shows how the AI Assistant (ChatGPT) and the Data Engineer communicate. Uploading the raw CSV dataset, creating the Medallion Architecture workflow, carrying out the Bronze, Silver, and Gold layers, verifying the quality of the data, producing reports, and managing the entire pipeline automation are all under the purview of the Data Engineer. In the meantime, ChatGPT serves as an AI-assisted support tool that assists the data engineer with code debugging, pipeline planning, data quality checking, and documentation enhancement. The automated pipeline execution includes the main processing stages from raw data ingestion in the Bronze layer, data cleaning and feature engineering in the Silver layer, business analytics generation in the Gold layer, and final quality validation before producing analytical outputs.

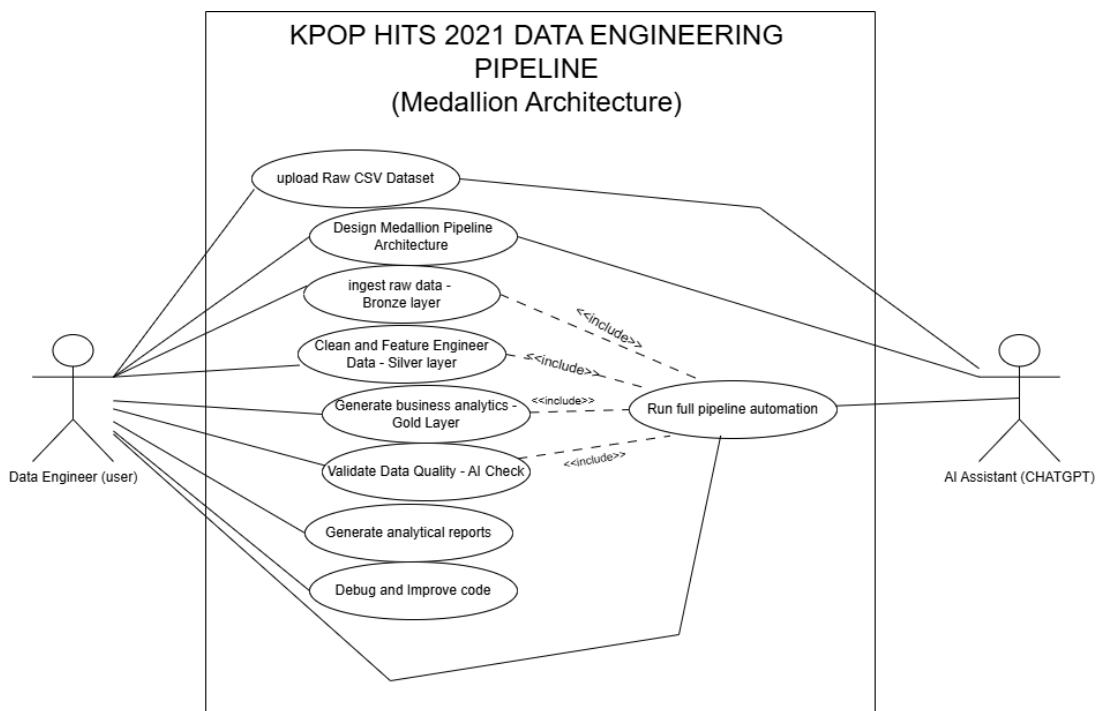


Figure 2: KPOP Hits 2021 Data Engineering Pipeline Use Case Diagram

d. Steps To Do Prompt And Coding

There will be two different coding steps included in this report since each member followed a different tutorial. Explanation will be done for both to understand what was done to produce the AI-assisted data pipelines.

1. Gemini (Google GenAI)

The development of our autonomous data pipeline was conducted systematically, dividing the engineering lifecycle into distinct operational phases within an integrated Google Colab environment. Rather than manually writing hardcoded data validation scripts, we engineered a dynamic framework that forces Gemini (via the modern Google GenAI library) to analyze an ingested profile and execute its own data remediation logic natively.

The final implemented pipeline follows a clear, production-grade Extract, Transform, and Load (ETL) sequential architecture:

Phase 1: Ingesting Unclean Data (Extract): Interactively fetches local datasets and applies fallback decoding exceptions to build a structured computational data profile. This phase acts as a defensive barrier at the entry point of your pipeline. Its primary objective is to pull an external, unstructured binary file into Python's volatile memory and extract it into a readable data frame, regardless of faulty formatting or legacy encodings. The first step is shown in Figure 2 where `files.upload()` opens an interactive browser widget allowing manual ingestion of a local storage file. Once uploaded, the raw data is captured purely as an unformatted sequence of binary bits (`file_bytes`).

```
print("--- PHASE 1: INGESTING UNCLEAR DATA ---")
# 1. Upload your local messy storage file
print("Please upload your unclean dataset (.csv or .txt):")
uploaded = files.upload()

if uploaded:
    filename = list(uploaded.keys())[0]
    file_bytes = uploaded[filename] # Get the byte content of the upload
```

Figure 2: Upload Raw File Code

Next, we apply a multi-tier exception handling hierarchy. The raw binary stream is encapsulated inside `io.BytesIO()` so that the Pandas engine can process it like an open file.

2. It first tests if the file is a compressed Microsoft Excel spreadsheet (`read_excel`).
3. If it catches an error, it treats it as a standard CSV decoded in modern UTF-8.

4. If it throws a `UnicodeDecodeError` (which happens when Excel exports special symbols using older Windows formats), it falls back to Latin-1 (cp1252) encoding. This triaging method avoids system crashes during early execution

This process is captured in Figure 3.

```
try:
    # Attempt 1: Try reading as an Excel file (common if .csv is mislab
    df_messy = pd.read_excel(io.BytesIO(file_bytes))
    print(f"Successfully read '{filename}' as an Excel file.")
except Exception as e_excel:
    print(f"Could not read '{filename}' as an Excel file. Error: {e_exc
    try:
        # Attempt 2: Try reading as CSV with utf-8 encoding
        df_messy = pd.read_csv(io.BytesIO(file_bytes), encoding='utf-8'
        print(f"Successfully read '{filename}' as a UTF-8 CSV file.")
    except UnicodeDecodeError as e_utf8:
        print(f"Could not read '{filename}' as a UTF-8 CSV file. Error:
        try:
            # Attempt 3: Try reading as CSV with latin1 encoding (commo
            df_messy = pd.read_csv(io.BytesIO(file_bytes), encoding='la
            print(f"Successfully read '{filename}' as a Latin-1 CSV fil
        except Exception as e_latin1:
            print(f"Could not read '{filename}' as a Latin-1 CSV file.
            # Fallback: If all specific attempts fail, raise a generic
            raise ValueError(f"Failed to read '{filename}'. It is neith
```

Figure 3: Multi-tier Exception Handling Code

After loading the data successfully, the pipeline summarizes the state of the dataset into a single text block called a Metadata Profile. It records column names, lengths, a text slice of the first 5 records, and a statistical aggregation of missing null values (`.isnull().sum()`) as shown in Figure 4.

```
# Print the messy data profile for the Agent to see
data_profile = f"""
Dataset Columns: {list(df_messy.columns)}
Row Count: {len(df_messy)}
Sample Data (First 5 rows):
{df_messy.to_string()}

Missing Values per column:
{df_messy.isnull().sum().to_string()}
"""

print("\n--- MESSY DATA RECEIVED ---")
print(df_messy)
```

Figure 4: Metadata Profile Code

Phase 2: AI Agent Analysis & Script Generation (Transform - Decision): Feeds the metadata profile into Gemini alongside strict prompt constraints to generate a standardized code payload. This phase represents the cognitive decision-making hub of the architecture. Instead of processing raw data matrix matrices directly, the AI Agent evaluates the high-level semantic metadata profile.

First, initialize the modern Google GenAI Client wrapper using the code shown in Figure 5. By passing the `api_key` directly into the object, the system securely grants the script programmatic authorization to connect to Google's cloud-hosted Gemini models.

```
client = genai.Client(api_key=api_key_secret)
```

Figure 5: Google GenAI Client Wrapper

Next, sets up strict operational constraints for the AI model as coded in Figure 6. Its function is to:

- `response_mime_type="application/json"`: Crucial for software engineering. It completely strips conversational text responses (like *"Sure, here is your code!"*) and forces Gemini to respond purely in structured JSON arrays that a machine can parse.
- `temperature=0.2`: A low value forces the LLM to behave deterministically, minimizing randomness or "hallucinations".

```
config=types.GenerateContentConfig(  
    response_mime_type="application/json",  
    temperature=0.2  
)
```

Figure 6: Operational Constraints for AI Model

The metadata profile is embedded directly inside an explicitly constrained engineering prompt instruction. The `gemini-2.5-flash` model reads it, diagnoses data quality issues such as unstandardized dates or lowercase strings, plans remediation steps, and outputs the result. The pipeline takes that raw JSON string and converts it back into an active Python dictionary using `json.loads()`. The process for these steps are captured in the code in Figure 7.

```
agent_prompt = f"""  
You are an expert AI Data Engineering Agent. Analyze the following profile  
Identify data quality issues (such as missing values, duplicate rows, non  
  
Write an automated, complete, and robust python code snippet using the pa  
  
CRITICAL IMPLEMENTATION RULE:  
1. Do not attempt to fix string text encoding errors using '.encode('lati  
2. For the 'date_added' column, you MUST standardize all various formats  
3. After parsing, you MUST explicitly format it to a uniform string date  
  
Data Profile:  
{data_profile}  
  
Your response must strictly be valid JSON matching this schema:  
{{  
    "analysis": "A brief explanation of the data quality issues found.",  
    "cleaning_code": "The raw Python code string to clean a dataframe named  
}}  
"""
```

```
try:  
    response = client.models.generate_content(  
        model='gemini-2.5-flash',  
        contents=agent_prompt,  
        config=types.GenerateContentConfig(  
            response_mime_type="application/json",  
            temperature=0.2  
        ),  
    )  
  
    # Parse the agent's decisions  
    agent_output = json.loads(response.text)
```

Figure 7: AI Prompt Processes

Phase 3: Pipeline Execution (Transform - Application): Copies the active memory dataframe and dynamically interprets the AI-generated instructions via a restricted execution loop (exec()). In this stage, the transformation strategy planned by the AI agent in Phase 2 is programmatically applied to your dataset.

The first step in this phase is an essential isolation technique in data engineering applied to the code in Figure 8. It clones the raw dataset in memory, ensuring that the original data source (df_messy) remains completely untouched and immutable. If an execution fails, the data is preserved safely.

```
df = df_messy.copy()
```

Figure 8: Raw Dataset Cloning

The pipeline then extracts the raw text string stored under the "cleaning_code" key from Gemini's JSON payload and feeds it to Python's internal exec() (Execute) function. The exec() statement takes a plain text string and converts it live into functional, operational Python code inside the system runtime. This allows the AI's cleaning rules (such as running multi-format date vector conversions like format='mixed') to execute dynamically on the isolated df object. This process is implemented in the code in Figure 9.

```
exec(agent_output['cleaning_code'])
```

Figure 9: Extracting Cleaning Code

Phase 4: Loading Clean Data (Load): Generates interactive analytical displays and outputs an immutable CSV artifact (agent_cleaned_output.csv) to local persistent storage. The final stage closes the ETL pipeline by presenting the finalized, structured dataset and saving it to local disk storage.

The script in Figure 10 calls Colab's interactive HTML rendering system (display(df)) to generate an aesthetic spreadsheet visual interface matrix, allowing stakeholders to visually inspect that the date formats, missing null blocks, and text strings have been completely standardized.

```
print("\n--- PHASE 4: LOADING CLEAN DATA ---")
print("✓ Pipeline executed successfully. Final Clean Dataframe:")
display(df)
```

Figure 10: Code to Show Pipeline Executed Successfully

Finally, the uniform dataframe is serialized and converted into an immutable comma-separated values (.csv) spreadsheet schema artifact named agent_cleaned_output.csv and written onto the physical drive block as coded in Figure

11. Setting `index=False` eliminates systemic numerical tracking row errors during subsequent imports.

```
df.to_csv("agent_cleaned_output.csv", index=False)  
print("\n✓ Saved agent-cleaned data to 'agent_cleaned_output.csv'")
```

Figure 11: Exporting Cleaned Dataset

2. ChatGPT (AI Assistant) Assisted K-pop Hits 2021 Data Pipeline Development

The development of the AI-Assisted K-pop Hits 2021 Data Pipeline was implemented using Python, Pandas, and Streamlit following the Medallion Architecture framework. Unlike traditional ETL pipelines where data moves directly from extraction to transformation, this project separates the data lifecycle into multiple layers:

- Bronze Layer: Raw Data Storage
- Silver Layer: Data Cleaning and Feature Engineering
- Gold Layer: Business Analytics
- AI Quality Check: Data Validation

ChatGPT was integrated as an AI Assistant during the development process to assist with pipeline design, debugging, code improvement, data quality validation logic, and feature engineering decisions.

Phase 1: Data Ingestion (Bronze Layer) : Ingesting the raw dataset into the Bronze Layer is the main goal of the K-pop Hits 2021 data pipeline's initial phase. The Bronze Layer serves as the first storage layer in a Medallion Architecture, where raw data is gathered and kept unaltered. This phase's goal is to produce a trustworthy replica of the original Kaggle dataset prior to any cleaning or transformation procedures.

Python Pandas was used to load the raw K-pop Hits 2021 CSV dataset for this research, which included music variables including song title, artist, danceability, energy, loudness, tempo, and duration. The Bronze Layer guarantees the preservation of the original dataset and offers a reliable source for the subsequent transformation phase.

Prompt Used:

```
I am developing an AI-Assisted K-pop Hits 2021 Data Engineering Pipeline using Medallion Architecture.

Create a Python Bronze Layer script using Pandas.

Requirements:
- Read raw CSV dataset from data/raw/KPopHits2021.csv
- Store an exact copy into data/bronze/kpop_sales_raw.csv
- Do not perform any cleaning or transformation
- Add validation to check whether file exists
- Display dataset rows, columns, column names, and sample data
- Include proper error handling
- Add comments explaining each section
```

Figure 12 : ChatGPT Prompt for Bronze Layer Development

Code Implementation

The Bronze Layer script (bronze.py) was developed to automate raw data ingestion. The script checks whether the input dataset exists, creates the Bronze directory automatically, loads the CSV file, displays dataset information, and saves the raw copy into the Bronze storage location.

The purpose of keeping the Bronze Layer unchanged is to maintain data traceability. If any issue occurs during later transformation stages, engineers can always return to the original raw dataset.

```
bronze.py X silver.py requirements.txt app.py README.md .gitignore
src > bronze.py > ...
1 # bronze.py
2 # Bronze Layer: Raw Data Ingestion (no transformation)
3
4 import os
5 import pandas as pd
6
7
8 INPUT_FILE = "data/raw/KPopHits2021.csv"
9 OUTPUT_FILE = "data/bronze/kpop_sales_raw.csv"
10
11
12 def run_bronze():
13
14     try:
15         if not os.path.exists(INPUT_FILE):
16             raise FileNotFoundError(f"Input file not found: {INPUT_FILE}")
17
18         os.makedirs(os.path.dirname(OUTPUT_FILE), exist_ok=True)
19
20         df = pd.read_csv(INPUT_FILE)
21
22         print("\n[Bronze Layer]")
23         print(f"Rows: {df.shape[0]} | Columns: {df.shape[1]}")
24
```

```

25     print("\nColumns:")
26     print(df.columns.tolist())
27
28     df.to_csv(OUTPUT_FILE, index=False)
29
30     print("\nBronze layer completed successfully.")
31     print(f"Saved: {OUTPUT_FILE}")
32
33     except Exception as e:
34         print("\nBronze Layer Error:", str(e))
35
36
37 if __name__ == "__main__":
38     run_bronze()

```

Figure 13 :Bronze Layer Code Implementation in VS Code

Bronze Layer Result

After generating the Bronze layer code using ChatGPT, the script was executed to ingest the original K-pop Hits 2021 dataset.

The Bronze layer successfully loaded the raw Kaggle dataset without applying any transformation or cleaning process. The purpose of this stage is to maintain the original dataset structure before any data processing occurs.

The execution result shows that the dataset contains:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  JUPYTER
PS C:\Users\LENOVO\Downloads\kpop-data-pipeline> python src\bronze.py

[Bronze Layer]
Rows: 50 | Columns: 14

Columns:
['Unnamed: 0', 'title', 'artist/s', 'danceability', 'energy', 'key', 'loudness', 'mode', 'speechiness', 'acousticness', 'valence', 'tempo',
'duration_ms', 'time_signature']

Bronze layer completed successfully.
Saved: data/bronze/kpop_sales_raw.csv

```

Figure 13 : Bronze Layer Execution Result

Phase 2: Data Transformation and Feature Engineering (Silver Layer) : The Bronze dataset is cleaned up and organized in the second stage. By eliminating duplicate entries, addressing missing values, standardizing text forms, correcting data kinds, and developing new analytical tools, the Silver Layer is in charge of enhancing data quality.

During this stage, synthetic characteristics were created because the original K-pop Hits 2021 dataset lacked business metrics like streaming count, sales count, and country region. The dataset can support analytical processing in the Gold Layer thanks to these extra features.

Prompt Used:

```
I have a Bronze Layer dataset from a K-pop Hits 2021 Kaggle dataset.

Create a Python Silver Layer script using Pandas.

Requirements:
- Read data/bronze/kpop_sales_raw.csv
- Remove duplicate rows
- Handle missing values
- Standardize text columns
- Fix incorrect data types
- Create synthetic analytical features:
  - streaming_count
  - sales_count
  - fan_preorder_count
  - country_region

Save the cleaned dataset into:
data/silver/kpop_sales_clean.csv

Include:
- validation checks
- comments
- error handling
```

Figure 14 : ChatGPT Prompt for Silver Layer Development

Code Implementation

The Silver Layer script (silver.py) cleans and enriches the raw K-pop dataset. The script improves dataset quality by applying transformation rules while maintaining the original data structure from Bronze Layer.

AI assistance was used during development to generate transformation logic, debug errors, and improve code structure. The AI acted as a coding assistant that supported the creation of reusable cleaning operations.

```
src > silver.py > ...
1  # silver.py
2  # Silver Layer: Cleaning + Feature Engineering
3
4  import os
5  import pandas as pd
6  import numpy as np
7
8
9  INPUT_FILE = "data/bronze/kpop_sales_raw.csv"
10 OUTPUT_FILE = "data/silver/kpop_sales_clean.csv"
11
12
13 def run_silver():
14
15     try:
16         if not os.path.exists(INPUT_FILE):
17             raise FileNotFoundError("Bronze file missing")
18
19         df = pd.read_csv(INPUT_FILE)
20
21         print("\n[Silver Layer]")
22         print(f"Original shape: {df.shape}")
23
24         # -----
25         # Clean duplicates
26         # -----
27         df = df.drop_duplicates()
28
```

```
src > silver.py > ...
13 def run_silver():
28
29     # -----
30     # Missing values
31     # -----
32     df.fillna({
33         col: "unknown" for col in df.select_dtypes(include="object").columns
34     }, inplace=True)
35
36     for col in df.select_dtypes(include=["number"]).columns:
37         df[col] = df[col].fillna(df[col].median())
38
39     # -----
40     # Standardize text
41     # -----
42     for col in df.select_dtypes(include="object").columns:
43         df[col] = df[col].str.lower().str.strip()
44
```

```

src > silver.py > ...
13 def run_silver():
45     # -----
46     # Feature Engineering
47     # -----
48     np.random.seed(42)
49     n = len(df)
50
51     df["streaming_count"] = np.random.randint(5000, 8000000, n)
52     df["sales_count"] = np.random.randint(1000, 2000000, n)
53
54     df["fan_preorder_count"] = (
55         df["sales_count"] * np.random.uniform(0.1, 0.4, n)
56     ).astype(int)
57
58     df["country_region"] = np.random.choice(
59         ["South Korea", "Japan", "USA", "Global", "SEA"], n
60     )
61
62     print("Cleaning completed")
63     print(f"Final shape: {df.shape}")
64
65     df.to_csv(OUTPUT_FILE, index=False)
66
67     print("Silver layer saved.")
68
69     except Exception as e:
70         print("Silver Layer Error:", str(e))
71
72
73 if __name__ == "__main__":
74     run_silver()

```

Figure 15 : Silver Layer Python Implementation Code

Silver Layer Result

After generating the Silver Layer code using ChatGPT, the script was executed to clean and transform the raw dataset from the Bronze Layer. This phase improved the dataset quality by applying data cleaning techniques and preparing the data for analytical processing.

The Silver Layer successfully removed duplicate records, handled missing values, standardized data formats, and generated additional analytical attributes required for business analysis. Since the original K-pop Hits 2021 dataset mainly contains music characteristics, synthetic features such as streaming count, sales count, fan preorder count, and country region were created to support the Gold Layer analysis.

The execution result shows that:


```

PS C:\Users\LENOVO\Downloads\kpop-data-pipeline> python src\silver.py

[Silver Layer]
Original shape: (50, 14)
C:\Users\LENOVO\Downloads\kpop-data-pipeline\src\silver.py:33: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to 'include' to select them, or to 'exclude' to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  col: "unknown" for col in df.select_dtypes(include="object").columns
C:\Users\LENOVO\Downloads\kpop-data-pipeline\src\silver.py:42: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to 'include' to select them, or to 'exclude' to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  for col in df.select_dtypes(include="object").columns:
Cleaning completed
Final shape: (50, 18)
Silver layer saved.

```

Figure 16 : Silver Layer Execution Result

Phase 3: Business Analytics Generation (Gold Layer) : The Gold Layer converts the cleaned Silver dataset into analytical datasets for reporting and decision-making. Instead of storing detailed records, this layer produces aggregated business insights.

The Gold Layer generates summaries such as:

- Artist performance
- Country analysis
- Top songs ranking
- Overall engagement metrics

These outputs are designed to support visualization through the Streamlit dashboard.

Prompt Used:

```

Create a Python Gold Layer script for my K-pop Hits 2021 data pipeline.

Input:
data/silver/kpop_sales_clean.csv

Requirements:
Generate analytical outputs:

1. Artist sales summary
2. Country sales summary
3. Top songs ranking
4. Engagement KPI summary

Save results into:
data/gold/

Include:
- Pandas aggregation
- sorting
- comments
- error handling

```

Figure 17 : ChatGPT Prompt for Gold Layer Development

Code Implementation

The Gold Layer (gold.py) applies aggregation techniques using Pandas groupby operations. The cleaned dataset is converted into business-friendly tables that can be used by dashboards and reports.

```
src > gold.py > ...
1  # gold.py
2  # Gold Layer: Business Analytics
3
4  import os
5  import pandas as pd
6
7
8  INPUT_FILE = "data/silver/kpop_sales_clean.csv"
9  OUTPUT_DIR = "data/gold"
10
11
12  def run_gold():
13
14      try:
15          if not os.path.exists(INPUT_FILE):
16              raise FileNotFoundError("Silver file missing")
17
18          os.makedirs(OUTPUT_DIR, exist_ok=True)
19
20          df = pd.read_csv(INPUT_FILE)
21
22          print("\n[Gold Layer]")
23          print(f"Rows: {df.shape[0]}")
24
25          artist_col = "artist/s"
26
```

```
src > gold.py > ...
12  def run_gold():
13
14      # -----
15      # Artist Summary
16      # -----
17      artist_summary = df.groupby(artist_col).agg(
18          total_sales=("sales_count", "sum"),
19          total_streams=("streaming_count", "sum"),
20          avg_streams=("streaming_count", "mean")
21      ).sort_values("total_sales", ascending=False)
22
23      artist_summary.to_csv(f"{OUTPUT_DIR}/artist_summary.csv")
24
25      # -----
26      # Country Summary
27      # -----
28      country_summary = df.groupby("country_region").agg(
29          total_sales=("sales_count", "sum"),
30          total_streams=("streaming_count", "sum")
31      ).sort_values("total_sales", ascending=False)
32
33      country_summary.to_csv(f"{OUTPUT_DIR}/country_summary.csv")
34
```

```

48 # -----
49 # Top Songs
50 # -----
51 top_songs = df.groupby("title").agg(
52     total_sales=("sales_count", "sum")
53 ).sort_values("total_sales", ascending=False)
54
55 top_songs.to_csv(f"{OUTPUT_DIR}/top_songs.csv")
56
57 print("Gold layer completed.")
58
59 except Exception as e:
60     print("Gold Layer Error:", str(e))
61
62
63 if __name__ == "__main__":
64     run_gold()

```

Figure 18 : Gold Layer Python Implementation Code

Gold Layer Result

After generating the Gold Layer code using ChatGPT, the script was executed to transform the cleaned Silver dataset into analytical outputs. The purpose of this stage is to convert processed data into meaningful business insights that can support reporting and visualization.

The Gold Layer applies aggregation techniques to summarize K-pop Hits 2021 data based on different analytical perspectives. Instead of storing individual song records, this layer creates business-ready datasets such as artist performance analysis, country distribution, top songs ranking, and overall engagement metrics.

The execution result confirms that the Gold Layer successfully generated:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  JUPYTER

PS C:\Users\LENOVO\Downloads\kpop-data-pipeline> python src\gold.py

[Gold Layer]
Rows: 50
Gold layer completed.

```

Figure 19 : Gold Layer Execution Result

Phase 4: AI Data Quality Validation : The final phase introduces AI-assisted quality checking. The purpose of this stage is to validate whether the processed Silver dataset meets expected data quality standards.

The AI Quality Check script automatically evaluates:

- missing values
- duplicate records
- empty columns
- incorrect data types
- invalid numerical values

AI improves this process by assisting engineers in identifying potential issues and suggesting improvements instead of manually checking every column.

Prompt Used:

```
Create an AI-assisted data quality checking Python script.

Dataset:
data/silver/kpop_sales_clean.csv

Check:

- missing values
- duplicate rows
- empty columns
- data types
- invalid numeric values

Generate:
- console quality report
- text report output

Add comments explaining the validation process.
```

Figure 20 : ChatGPT Prompt for Quality Check Development

Code Implementation

The AI Quality Check module validates the final cleaned dataset before analytics usage. This improves reliability because incorrect or incomplete data can be detected before reaching reporting systems.

```

src > ai_quality_check.py > ...
15 def run_quality_check():
101
102     with open(REPORT_FILE, "w", encoding="utf-8") as f:
103         f.write("\n".join(report_lines))
104
105     print("\n=== QUALITY CHECK COMPLETED ===")
106     print(f"Report saved to: {REPORT_FILE}")
107
108     except FileNotFoundError as fnf:
109         print("\n❌ File Error:", fnf)
110
111     except Exception as e:
112         print("\n❌ Unexpected Error:", str(e))
113
114
115 # -----
116 # Run check
117 # -----
118 if __name__ == "__main__":
119     run_quality_check()

```

Figure 21 : AI Quality Check Python Implementation Code

AI Quality Check Result

After generating the AI Quality Check script using ChatGPT, the validation process was executed on the Silver Layer dataset. This phase ensures that the cleaned dataset meets the required data quality standards before being used for analytics and reporting.

The AI-assisted quality checking process automatically examined different quality dimensions including missing values, duplicate records, empty columns, data types, and invalid numerical values. This reduces manual inspection effort and improves reliability during pipeline execution.

The execution result shows that:

```
PS C:\Users\LENOVO\Downloads\kpop-data-pipeline> python src\ai_quality_check.py

=== AI DATA QUALITY CHECK STARTED ===
Rows: 50
Columns: 18

Missing Values:
  Unnamed: 0      0
title            0
artist/s         0
danceability     0
energy           0
key              0
loudness         0
mode             0
speechiness      0
acousticness     0
valence          0
tempo            0
duration_ms      0
time_signature   0
streaming_count  0
sales_count      0
fan_preorder_count 0
country_region   0
dtype: int64

Duplicate Rows: 0

Empty Columns: []

Empty Columns: []

Data Types:
  Unnamed: 0      int64
title           str
artist/s        str
danceability    float64
energy          float64
key             int64
loudness        float64
mode            int64
speechiness     float64
acousticness    float64
valence         float64
tempo           float64
duration_ms     int64
time_signature  int64
streaming_count int64
sales_count     int64
fan_preorder_count int64
country_region  str
dtype: object

Invalid Numeric Values (Negative Check):
{'Unnamed: 0': 0, 'danceability': 0, 'energy': 0, 'key': 0, 'loudness': 50, 'mode': 0, 'speechiness': 0, 'acousticness': 0, 'valence': 0, 'tempo': 0, 'duration_ms': 0, 'time_signature': 0, 'streaming_count': 0, 'sales_count': 0, 'fan_preorder_count': 0}

=== QUALITY CHECK COMPLETED ===
Report saved to: data/gold/data quality report.txt
```

Figure 22 : AI Quality Check Execution Result

Data_quality_report.txt

```
nts.txt  app.py  README.md  data > gold > data_quality_report.txt
1  === MISSING VALUES ===
2  Unnamed: 0 0
3  title 0
4  artist/s 0
5  danceability 0
6  energy 0
7  key 0
8  loudness 0
9  mode 0
10 speechiness 0
11 acousticness 0
12 valence 0
13 tempo 0
14 duration_ms 0
15 time_signature 0
16 streaming_count 0
17 sales_count 0
18 fan_preorder_count 0
19 country_region 0
20 dtype: int64
21
22 === DUPLICATES ===
23 Total duplicate rows: 0
24
```

```
22 === DUPLICATES ===
23 Total duplicate rows: 0
24
25 === EMPTY COLUMNS ===
26 []
27
```

```

28  === DATA TYPES ===
29  Unnamed: 0          int64
30  title               str
31  artist/s           str
32  danceability        float64
33  energy              float64
34  key                 int64
35  loudness            float64
36  mode                int64
37  speechiness         float64
38  acousticness        float64
39  valence             float64
40  tempo               float64
41  duration_ms         int64
42  time_signature       int64
43  streaming_count      int64
44  sales_count         int64
45  fan_preorder_count  int64
46  country_region      str
47  dtype: object

```

```

48
49  === INVALID NUMERIC VALUES (NEGATIVE CHECK) ===
50  {'Unnamed: 0': 0, 'danceability': 0, 'energy': 0, 'key': 0, 'loudness': 50, 'mode': 0,
51   'speechiness': 0, 'acousticness': 0, 'valence': 0, 'tempo': 0, 'duration_ms': 0,
52   'time_signature': 0, 'streaming_count': 0, 'sales_count': 0, 'fan_preorder_count': 0}
53
54  === SUMMARY ===
55  Total rows: 50
56  Total columns: 18
57  Duplicate rows: 0

```

Figure 23 : AI Quality Check saved in Report

Phase 5: Dashboard Visualization (Streamlit Application) : After completing the Bronze, Silver, Gold, and AI Quality Check phases, a Streamlit dashboard was developed to visualize the processed K-pop Hits 2021 dataset. The purpose of this dashboard is to provide an interactive interface where users can explore the pipeline outputs without directly opening CSV files.

The application reads the analytical datasets generated by the Gold Layer and presents important insights through KPI cards, tables, and charts. This allows stakeholders to understand song performance, artist trends, country distribution, and engagement metrics generated from the data pipeline.

The Streamlit application acts as the presentation layer of the architecture, connecting the data engineering pipeline with user-friendly analytics visualization.

Prompt Used:

```
I have created an AI-Assisted K-pop Hits 2021 Data Engineering Pipeline using Medallion Architecture.

Create a Streamlit dashboard application using Python.

Requirements:

- Load datasets from:
data/raw/KPopHits2021.csv
data/silver/kpop_sales_clean.csv
data/gold/

Create dashboard sections:
↓

1. Overview KPI Dashboard
2. Raw Data Preview
3. Silver Clean Dataset Preview
4. Gold Analytics Visualization

Display:
- total sales
- total streaming
- fan preorder count
- top artists
- country analysis
- top songs ranking

Use:
- pandas
- streamlit
↓

Include:
- error handling
- comments
- clean structure
```

Figure 24 : ChatGPT Prompt for Visualization Development

Code Implementation

The Streamlit application (app.py) was created to provide a visual representation of the completed data pipeline. The application loads the datasets generated from different layers and organizes the information into several sections.

The dashboard contains four main sections:

1. **Overview Dashboard**

Displays key performance indicators such as total sales, total streams, and preorder metrics generated from the Gold Layer.

2. **Raw Data Preview**

Allows users to view the original Kaggle dataset before any processing occurs in the Bronze Layer.

3. **Silver Data Preview**

Displays the cleaned and enriched dataset after transformation, including generated analytical attributes.

4. **Gold Analytics**

Presents business insights using charts and tables, including top artists, country analysis, and song performance ranking.

AI assistance was used during development to generate the initial Streamlit structure, debug data loading errors, and improve the dashboard organization.

```

app.py > ...
1  import streamlit as st
2  import pandas as pd
3  from pathlib import Path
4
5  # =====
6  # PAGE CONFIG
7  # =====
8
9  st.set_page_config(
10     page_title="K-pop Hits 2021 Dashboard",
11     page_icon="🎵",
12     layout="wide"
13 )
14
15 # =====
16 # TITLE
17 # =====
18
19 st.title("🎵 K-pop Hits 2021 Dashboard")
20
21 st.markdown("""
22 AI-Assisted Pipeline using Medallion Architecture:
23 Bronze → Silver → Gold
24 """)
25

```

```

26 # =====
27 # FILE PATHS
28 # =====
29
30 RAW_FILE = Path("data/raw/KPopHits2021.csv")
31 SILVER_FILE = Path("data/silver/kpop_sales_clean.csv")
32
33 ARTIST_FILE = Path("data/gold/artist_summary.csv")
34 COUNTRY_FILE = Path("data/gold/country_summary.csv")
35 TOP_SONGS_FILE = Path("data/gold/top_songs.csv")
36 ENGAGEMENT_FILE = Path("data/gold/engagement_summary.csv")
37
38 # =====
39 # LOAD DATA SAFELY
40 # =====
41
42 @st.cache_data
43 def load_data():
44     raw_df = pd.read_csv(RAW_FILE)
45     silver_df = pd.read_csv(SILVER_FILE)
46
47     artist_df = pd.read_csv(ARTIST_FILE)
48     country_df = pd.read_csv(COUNTRY_FILE)
49     top_songs_df = pd.read_csv(TOP_SONGS_FILE)
50     engagement_df = pd.read_csv(ENGAGEMENT_FILE)
51
52     return raw_df, silver_df, artist_df, country_df, top_songs_df, engagement_df
53

```

```

55 try:
56     raw_df, silver_df, artist_df, country_df, top_songs_df, engagement_df = load_data()
57
58 except Exception as e:
59     st.error(f"Data loading error: {e}")
60     st.stop()
61
62 # =====
63 # SIDEBAR
64 # =====
65
66 section = st.sidebar.radio(
67     "Navigation",
68     ["Overview", "Raw Data", "Silver Data", "Gold Analytics"]
69 )
70
71 # =====
72 # OVERVIEW (FIXED KPI LOGIC)
73 # =====
74
75 if section == "Overview":
76
77     st.header("📊 KPI Dashboard")
78
79     # FIX: engagement is metric-value format
80     engagement_map = dict(zip(
81         engagement_df["metric"],
82         engagement_df["value"]
83     ))
84

```

```

84
85     total_sales = int(engagement_map.get("total_sales", 0))
86     total_streams = int(engagement_map.get("total_streams", 0))
87     total_preorders = int(engagement_map.get("total_preorders", 0))
88
89     col1, col2, col3 = st.columns(3)
90
91     col1.metric("Total Sales", f"{total_sales:,}")
92     col2.metric("Total Streams", f"{total_streams:,}")
93     col3.metric("Total Preorders", f"{total_preorders:,}")
94
95     st.markdown("---")
96
97     st.write("Pipeline: Bronze → Silver → Gold → AI Quality Check")
98
99 # =====
100 # RAW DATA
101 # =====
102
103 elif section == "Raw Data":
104
105     st.header("📄 Raw Dataset")
106
107     st.write(f"Rows: {raw_df.shape[0]} | Columns: {raw_df.shape[1]}")
108     st.dataframe(raw_df)
109

```

```

110 # =====
111 # SILVER DATA
112 # =====
113
114 elif section == "Silver Data":
115
116     st.header("🧹 Cleaned Dataset")
117
118     st.write(f"Rows: {silver_df.shape[0]} | Columns: {silver_df.shape[1]}")
119     st.dataframe(silver_df)
120
121 # =====
122 # GOLD ANALYTICS
123 # =====
124
125 elif section == "Gold Analytics":
126
127     st.header("💡 Business Insights")

```

```

129 # -----
130 # Top Artists
131 # -----
132
133 st.subheader("Top Artists by Sales")
134
135 artist_chart = artist_df.sort_values("total_sales", ascending=False).head(10)
136
137 # FIX: artist column name safe handling
138 artist_col = "artist/s" if "artist/s" in artist_df.columns else artist_df.columns[0]
139
140 st.bar_chart(
141     artist_chart.set_index(artist_col)["total_sales"]
142 )
143
144 st.dataframe(artist_chart)
145
146 st.markdown("---")

```

```

148 # -----
149 # Countries
150 # -----
151
152 st.subheader("Top Countries by Sales")
153
154 country_chart = country_df.sort_values("total_sales", ascending=False).head(10)
155
156 st.bar_chart(
157     country_chart.set_index("country_region")["total_sales"]
158 )
159
160 st.dataframe(country_chart)
161
162 st.markdown("---")
163
164 # -----
165 # Top Songs
166 # -----
167
168 st.subheader("Top Songs by Sales")
169
170 songs_chart = top_songs_df.sort_values("total_sales", ascending=False).head(10)
171
172 st.bar_chart(
173     songs_chart.set_index("title")["total_sales"]
174 )
175
176 st.dataframe(songs_chart)

```

```

178 # =====
179 # FOOTER
180 # =====
181
182 st.markdown("---")
183
184 st.caption(
185     "K-pop Data Engineering Pipeline | Built with Python, Pandas & Streamlit"
186 )

```

Figure 25 : Streamlit Dashboard Python Implementation Code

Dashboard Execution Result

After completing the Streamlit application, the dashboard was executed using the Streamlit command. The application successfully loaded the processed datasets from the pipeline and displayed the analytical results through an interactive dashboard interface.

The dashboard demonstrates that the AI-assisted data engineering pipeline successfully transforms raw K-pop Hits 2021 data into meaningful visual insights through the complete workflow:

Bronze → Silver → Gold → Dashboard

The displayed results include:

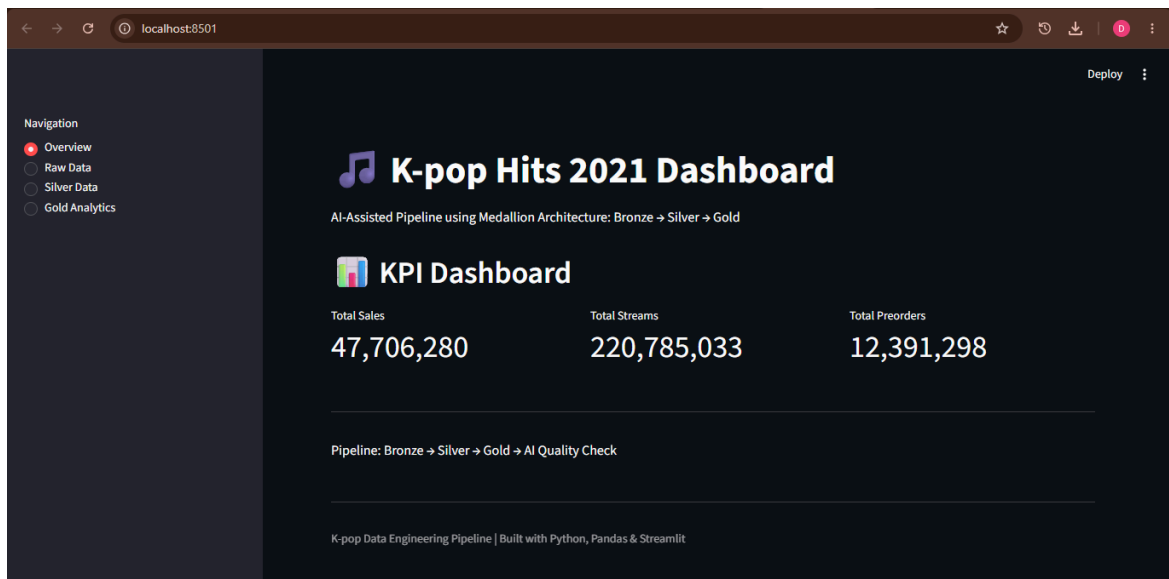


Figure 26 : Dashboard Overview

Navigation

- Overview
- Raw Data**
- Silver Data
- Gold Analytics

Raw Dataset

Rows: 50 | Columns: 14

	Unnamed: 0	title	artist/s	danceability	energy	key	loudness	mode	speechiness	acousticness	valence
0	0	Butter	BTS	0.759	0.459	8	-5.187	1	0.0948	0.0032	0.695
1	1	The Feels	TWICE	0.808	0.903	9	-2.179	1	0.0729	0.0899	0.922
2	2	Savage	aespa	0.727	0.879	9	-1.167	1	0.129	0.124	0.671
3	3	Permission to Dance	BTS	0.702	0.741	9	-5.33	1	0.0427	0.0054	0.646
4	4	In the morning	ITZY	0.846	0.814	6	-4.685	0	0.149	0.0823	0.722
5	5	Alcohol-Free	TWICE	0.688	0.909	8	-1.992	1	0.0624	0.0474	0.702
6	6	MONEY	LISA	0.826	0.553	1	-10.121	0	0.234	0.163	0.4
7	7	ELEVEN	IVE	0.828	0.731	9	-3.033	1	0.111	0.0617	0.587
8	8	Queendom	Red Velvet	0.695	0.896	5	-1.24	1	0.0847	0.179	0.508
9	9	Celebrity	IU	0.73	0.784	9	-2.62	1	0.0691	0.191	0.673

Figure 27 : Raw Data (Bronze Layer) Dashboard

Navigation

- Overview
- Raw Data
- Silver Data**
- Gold Analytics

Cleaned Dataset

Rows: 50 | Columns: 18

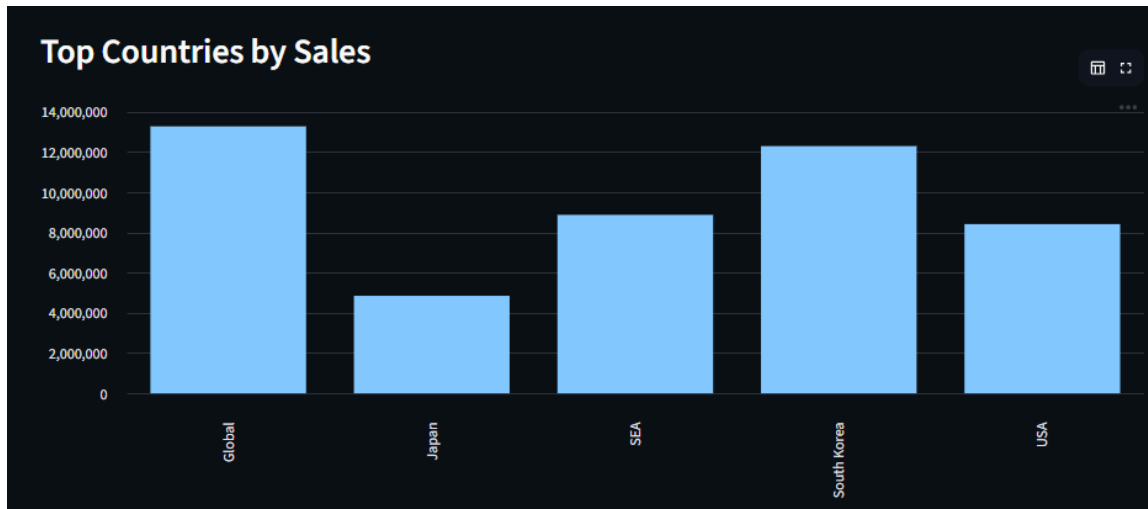
	Unnamed: 0	title	artist/s	danceability	energy	key	loudness	mode	speechiness	acousticness	valence
0	0	butter	bts	0.759	0.459	8	-5.187	1	0.0948	0.0032	0.695
1	1	the feels	twice	0.808	0.903	9	-2.179	1	0.0729	0.0899	0.922
2	2	savage	aespa	0.727	0.879	9	-1.167	1	0.129	0.124	0.671
3	3	permission to dance	bts	0.702	0.741	9	-5.33	1	0.0427	0.0054	0.646
4	4	in the morning	itzy	0.846	0.814	6	-4.685	0	0.149	0.0823	0.722
5	5	alcohol-free	twice	0.688	0.909	8	-1.992	1	0.0624	0.0474	0.702
6	6	money	lisa	0.826	0.553	1	-10.121	0	0.234	0.163	0.4
7	7	eleven	ive	0.828	0.731	9	-3.033	1	0.111	0.0617	0.587
8	8	queendom	red velvet	0.695	0.896	5	-1.24	1	0.0847	0.179	0.508
9	9	celebrity	iu	0.73	0.784	9	-2.62	1	0.0691	0.191	0.673

Figure 28 : Cleaned Dataset (Silver Layer) Dashboard



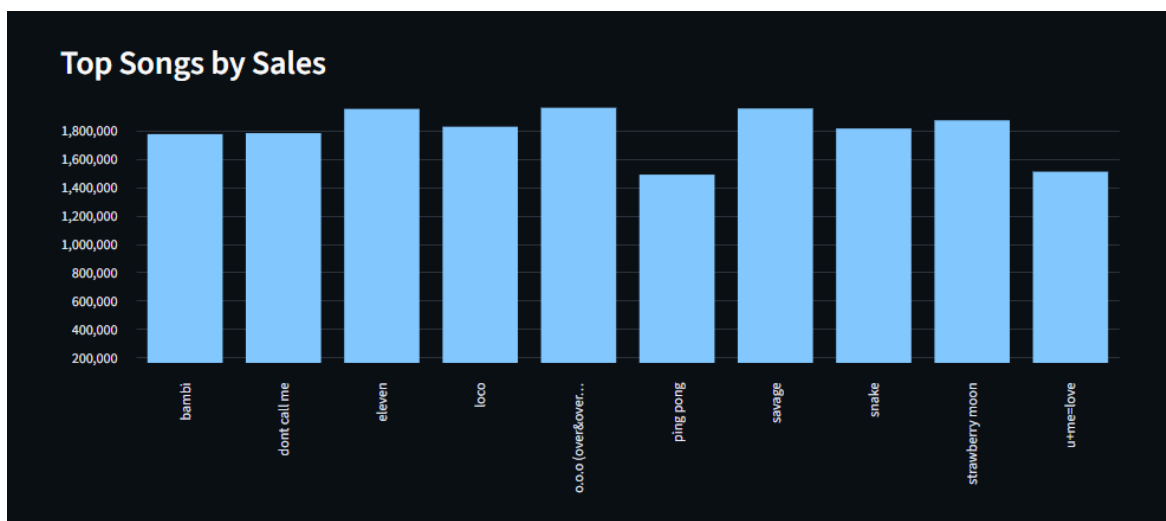
	artist/s	total_sales	total_streams	avg_streams
0	twice	3799489	14235113	4745037.6667
1	iu	2771058	10597183	3532394.3333
2	tomorrow x together	2633051	7197446	2399148.6667
3	itzy	2146695	11082007	5541003.5
4	girls planet 999	1960366	7580828	7580828
5	nct dream	1957220	13729997	4576665.6667
6	aespa	1955354	6428388	6428388
7	ive	1952224	4931484	4931484
8	medusa	1814045	7494535	7494535
9	shinee	1781488	4986505	4986505

Figure 29 : Business Analytics (Gold Layer) - Top Artist by Sales



	country_region	total_sales	total_streams
0	Global	13275270	74376781
1	South Korea	12294234	53845616
2	SEA	8875264	32629926
3	USA	8413795	42586039
4	Japan	4847717	17346671

Figure 30 : Business Analytics (Gold Layer) - Top Countries by Sales



	title	total_sales
0	o.o.o (over&over&over)	1960366
1	savage	1955354
2	eleven	1952224
3	strawberry moon	1871928
4	loco	1826665
5	snake	1814045
6	dont call me	1781488
7	bambi	1774415
8	u+me=love	1509349
9	ping pong	1488550

Figure 31 : Business Analytics (Gold Layer) - Top Songs by Sales

e. Reflection

Nurul Ika Syafiny Binti Azhar

This tutorial successfully demonstrates how shifting from rigid, rule-based coding to an autonomous AI agent workflow can revolutionize traditional data engineering tasks. By integrating the Gemini API directly into our Python pipeline, we moved past using AI as a basic code copy-paste tool and instead built an adaptive system that can independently audit a dataset's metadata, diagnose structural flaws, and execute its own custom remediation code at runtime. Overcoming real-world pipeline failures such as handling complex international text encoding bugs and parsing messy, multi-format Netflix date strings highlighted the vital importance of prompt engineering and structured JSON guardrails to ensure system predictability. Ultimately, this agentic approach delivers immense scalability for modern enterprise data systems, showing that while AI introduces non-deterministic variations, it can be safely constrained by software engineering guardrails to clean messy data autonomously without human intervention.

Damiya Aina binti Basir Abd Shammad

The development of the AI-assisted K-pop Hits 2021 data pipeline shows how integrating artificial intelligence tools like ChatGPT can greatly improve contemporary data engineering techniques. All phases of the Medallion Architecture, including Bronze, Silver, Gold, and the AI Quality Check layer, employed AI not only as a coding help but also as a design and debugging collaborator. The way raw data moves through a structured pipeline beginning with ingestion in the Bronze Layer, cleaning and feature engineering in the Silver Layer, and ultimately turning into analytical insights in the Gold Layer became more apparent to the developer. By helping with error correction, enhancing code structure, and producing artificial business features when the dataset lacked real-world metrics like sales and streaming counts, the application of AI decreased development complexity.

In addition, the integration of AI improved problem-solving efficiency during debugging stages. Instead of manually troubleshooting errors, the developer was able to use AI suggestions to quickly identify issues related to column mismatches, missing files, and incorrect aggregations. This allowed faster iteration and improved overall pipeline stability. The Streamlit dashboard further strengthened the understanding of end-to-end data engineering by visualizing the final outputs in an interactive format. This helped connect technical processing with real-world business interpretation, such as identifying top-performing artists and countries in the K-pop industry. To summarize, the research shows that AI-assisted data engineering is an augmentation tool that increases efficiency, accuracy, and scalability in contemporary data processes rather than a substitute for traditional engineering abilities.

References

Bansal, S. (2021). *Netflix Movies and TV Shows*. Wwww.kaggle.com.

<https://www.kaggle.com/datasets/shivamb/netflix-shows>

Idan Novogroder. (2024, December 16). *AI in Data Engineering: Challenges, Best Practices & Tools*. Git for Data - LakeFS. <https://lakefs.io/blog/ai-data-engineering/>

Berjamin, S. A. (n.d.). *K-Pop Hits Through the Years*. [Wwww.kaggle.com](https://www.kaggle.com).

<https://www.kaggle.com/datasets/sberj127/kpop-hits-through-the-years?resource=download&select=KPopHits2021.csv>